PYTHON PANDAS
THE COMPLETE PRACTICAL GUIDE

A Comprehensive, Line-by-Line Reference Guide

From Series and DataFrames to GroupBy, Merging, Reshaping, Time Series and Performance

---

Kajola Gbenga
Data Scientist & AI Engineer

Table of Contents

1. Introduction to Pandas
2. Setting Up the Environment
3. Core Data Structures: Series and DataFrame
4. Creating Series and DataFrames
5. Loading Data From Files
6. Initial Data Inspection
7. Selecting and Indexing Data
8. Filtering and Boolean Indexing
9. Handling Missing Data
10. Handling Duplicate Records
11. Data Type Conversion
12. String Operations
13. Applying Functions and Vectorization
14. Sorting and Ranking
15. Grouping and Aggregation (GroupBy)
16. Merging, Joining and Concatenating
17. Reshaping Data (Pivot, Melt, Stack/Unstack)
18. MultiIndex (Hierarchical Indexing)
19. Time Series Handling
20. Window Functions (Rolling, Expanding, EWM)
21. Categorical Data and Memory Optimization
22. Exporting Data
23. Performance Tips and Best Practices
24. Full Worked Example (Line-by-Line, End-to-End)
25. Common Pitfalls and How to Avoid Them
26. Quick-Reference Cheat Sheet
27. Conclusion

1. Introduction to Pandas

1.1 What is Pandas?

Pandas is the standard Python library for working with structured, tabular data. It provides two core data structures, the Series (a single labeled column of data) and the DataFrame (a labeled, two-dimensional table), along with thousands of methods for reading, cleaning, transforming, aggregating and writing data. The name 'pandas' is derived from 'panel data', a term from econometrics describing multi-dimensional datasets that track many entities over time.

Pandas was created by Wes McKinney in 2008 and has since become the backbone of the Python data science ecosystem. It sits underneath or alongside almost every other data tool in Python: scikit-learn accepts pandas DataFrames for machine learning, matplotlib and seaborn plot directly from them, and most data loading pipelines produce a DataFrame as their final output.

1.2 Why Pandas Matters

- It replaces slow, manual loops over data with fast, vectorized operations implemented in C under the hood.
- It provides a consistent, labeled way to reference rows and columns, instead of tracking raw numeric array positions.
- It handles messy real-world data gracefully, with built-in support for missing values, mixed types and heterogeneous columns.
- It offers a huge library of built-in operations: filtering, grouping, merging, reshaping, resampling and more, each usually a single method call.
- It reads and writes virtually every common data format: CSV, Excel, JSON, SQL, Parquet, HTML tables and more.

1.3 Series vs DataFrame at a Glance

A Series is a one-dimensional labeled array, essentially a single column with an index. A DataFrame is a two-dimensional labeled table, essentially a collection of Series that share the same index, arranged side by side as columns. Understanding this relationship, a DataFrame is built from Series, is the single most useful mental model for learning pandas, since almost every DataFrame operation is really a coordinated operation across its underlying Series.

1.4 How This Guide Is Organized

This guide moves from the fundamentals (creating and inspecting data) through cleaning, transformation, aggregation, reshaping, time series and performance, ending with a full worked example that ties every technique together. For the most important code blocks, a line-by-line breakdown table explains exactly what each line does, in the same format used throughout.

> **How to use this guide:** Every code example below assumes pandas has already been imported as pd and numpy as np, as shown in section 2, unless the example itself contains the import.

2. Setting Up the Environment

2.1 Installing Pandas

```
pip install pandas numpy openpyxl xlrd pyarrow sqlalchemy
```

Code Line	Explanation
`pandas`	The core library itself.
`numpy`	The numerical array library pandas is built on; needed for functions like np.nan and np.where.
`openpyxl`	A dependency required by pandas to read and write modern .xlsx Excel files.
`xlrd`	A dependency required for reading legacy .xls Excel files.
`pyarrow`	Enables fast reading and writing of the Parquet columnar file format, and powers pandas' newer Arrow-backed data types.
`sqlalchemy`	A toolkit that lets pandas connect to and query relational databases such as PostgreSQL, MySQL or SQLite.

2.2 The Standard Import Block

```
import pandas as pd
import numpy as np
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
pd.set_option('display.width', 120)
pd.set_option('display.float_format', lambda x: f'{x:,.2f}')
```

Code Line	Explanation
`import pandas as pd`	Imports pandas under the conventional short alias pd, used throughout the rest of this guide.
`import numpy as np`	Imports numpy under its conventional alias np, since pandas relies on it for numeric arrays, NaN handling, and mathematical functions.
`pd.set_option('display.max_columns', None)`	Removes the limit on how many columns are shown when printing a wide DataFrame; None means unlimited.
`pd.set_option('display.max_rows', 100)`	Caps the number of rows shown when printing a DataFrame at 100, preventing very long tables from flooding the output.
`pd.set_option('display.width', 120)`	Sets the total character width used when displaying a DataFrame in a terminal or notebook, controlling line wrapping.
`pd.set_option('display.float_format', lambda x: f'{x:,.2f}')`	Applies a custom formatting function to every floating point number displayed, here showing two decimal places with a thousands separator, for example 1,234.50 instead of 1234.5.

2.3 Checking the Installed Version

```
print(pd.__version__)
```

Pandas has evolved significantly across major versions (notably the 1.x to 2.x transition, which introduced the optional PyArrow backend and copy-on-write semantics). Checking the version explains why a method might behave slightly differently between two machines or tutorials.

3. Core Data Structures: Series and DataFrame

3.1 The Series

```
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'], name='scores')
print(s)
```

Code Line	Explanation
`pd.Series([10, 20, 30, 40], ...)`	Constructs a new Series from a plain Python list of values; each value will occupy one labeled position.
`index=['a', 'b', 'c', 'd']`	Assigns custom labels to each position instead of the default 0, 1, 2, 3 integer index; the index is what lets pandas align values by label rather than raw position.
`name='scores'`	Gives the Series itself a name, which becomes useful when the Series is later converted into a column of a DataFrame.

Every Series has three core components: the values (a numpy or Arrow-backed array), the index (the labels attached to each value), and an optional name. Internally, a Series is essentially a numpy array with a label attached to each element.

3.2 The DataFrame

```
df = pd.DataFrame({
    'name': ['Ada', 'Grace', 'Alan'],
    'age': [36, 85, 41],
    'field': ['Mathematics', 'Computer Science', 'Computer Science']
})
print(df)
```

Code Line	Explanation
`pd.DataFrame({...})`	Constructs a DataFrame from a Python dictionary, where each key becomes a column name and each associated list becomes that column's values.
`'name': ['Ada', 'Grace', 'Alan']`	Defines the 'name' column with three string values, one per row.
`Each key-value pair`	Every dictionary entry must have the same number of values, since a DataFrame requires every column to be the same length (share the same row index).

3.3 Key Attributes

```
df.shape
df.columns
df.index
df.values
df.dtypes
df.size
df.ndim
```

Code Line	Explanation
`df.shape`	A tuple of (number_of_rows, number_of_columns).
`df.columns`	An Index object listing the column names, in order.
`df.index`	An Index object listing the row labels; by default a RangeIndex of 0, 1, 2, ... but can be replaced with dates, IDs or any other label.
`df.values`	Returns the underlying data as a plain numpy array, stripped of row and column labels.
`df.dtypes`	A Series listing the data type of every column.
`df.size`	The total number of elements in the DataFrame, equal to rows multiplied by columns.
`df.ndim`	The number of dimensions of the object; 1 for a Series, 2 for a DataFrame.

4. Creating Series and DataFrames

4.1 Creating a Series From Different Sources

```
s1 = pd.Series([1, 2, 3])
s2 = pd.Series({'a': 1, 'b': 2, 'c': 3})
s3 = pd.Series(5, index=['x', 'y', 'z'])
s4 = pd.Series(np.arange(5))
```

Code Line	Explanation
`pd.Series([1, 2, 3])`	Creates a Series from a list, receiving a default integer index of 0, 1, 2.
`pd.Series({'a': 1, 'b': 2, 'c': 3})`	Creates a Series from a dictionary; the dictionary keys automatically become the index labels, and the values become the Series data.
`pd.Series(5, index=['x', 'y', 'z'])`	Broadcasts a single scalar value (5) across every position defined by the given index, producing a Series of three 5s.
`pd.Series(np.arange(5))`	Creates a Series from a numpy array; np.arange(5) generates the array [0, 1, 2, 3, 4].

4.2 Creating a DataFrame From Different Sources

```
df1 = pd.DataFrame({'a': [1, 2], 'b': [3, 4]})
df2 = pd.DataFrame([[1, 3], [2, 4]], columns=['a', 'b'])
df3 = pd.DataFrame([{'a': 1, 'b': 3}, {'a': 2, 'b': 4}])
df4 = pd.DataFrame(np.random.randint(0, 100, (5, 3)), columns=['x', 'y', 'z'])
```

Code Line	Explanation
`pd.DataFrame({'a': [1, 2], 'b': [3, 4]})`	Column-oriented construction: a dictionary of lists, where each key becomes a column.
`pd.DataFrame([[1, 3], [2, 4]], columns=['a', 'b'])`	Row-oriented construction: a list of lists, where each inner list is one row; the columns argument supplies the column names since a plain list of lists carries no labels.
`pd.DataFrame([{'a': 1, 'b': 3}, {'a': 2, 'b': 4}])`	Constructs a DataFrame from a list of dictionaries, where each dictionary represents one row and its keys become column names; this pattern is common when data comes from an API returning JSON records.
`pd.DataFrame(np.random.randint (0, 100, (5, 3)), columns=['x', 'y', 'z'])`	Builds a DataFrame directly from a numpy array of random integers between 0 and 100, shaped as 5 rows by 3 columns, with explicit column names supplied separately since a raw numpy array has none.

4.3 Empty and Placeholder DataFrames

```
df_empty = pd.DataFrame(columns=['id', 'name', 'value'])
df_nan = pd.DataFrame(np.nan, index=range(3), columns=['a', 'b'])
```

df_empty creates a DataFrame with defined column names but zero rows, useful as a template to append results into during a loop. df_nan pre-allocates a DataFrame filled entirely with NaN of a known shape, which can then be filled in progressively; this is often faster than growing a DataFrame row by row inside a loop.

5. Loading Data From Files

5.1 Reading CSV Files

```
df = pd.read_csv(
    'employees.csv',
    sep=',',
    header=0,
    usecols=['id', 'name', 'department', 'salary', 'hire_date'],
    parse_dates=['hire_date'],
    dtype={'id': str},
    na_values=['NA', 'N/A', 'null'],
    nrows=10000
)
```

Code Line	Explanation
`sep=','`	The delimiter separating values in each line; change to ';' or '\t' for semicolon- or tab-delimited files.
`header=0`	Tells pandas the first row (row index 0) holds the column names.
`usecols=[...]`	Loads only the listed columns, skipping the rest during parsing, which is faster and uses less memory for wide files where only some columns are needed.
`parse_dates=['hire_date']`	Automatically converts the hire_date column from text into pandas datetime objects while reading.
`dtype={'id': str}`	Forces the id column to be read as text, preventing pandas from auto-detecting it as a number and stripping leading zeros.
`na_values=['NA', 'N/A', 'null']`	Adds extra strings that should be interpreted as missing values on top of pandas' built-in defaults.
`nrows=10000`	Limits reading to only the first 10,000 rows, useful for quickly previewing a huge file without loading it in full.

5.2 Reading Excel Files

```
df = pd.read_excel('report.xlsx', sheet_name='Q3', skiprows=2, engine='openpyxl')
all_sheets = pd.read_excel('report.xlsx', sheet_name=None)
```

Code Line	Explanation
`sheet_name='Q3'`	Reads only the worksheet named 'Q3'; can also be an integer position such as 0 for the first sheet.
`skiprows=2`	Skips the first 2 rows of the file before parsing begins, useful when a report has title rows above the actual header.
`engine='openpyxl'`	Explicitly selects the engine used to parse the file; openpyxl handles modern .xlsx files, while legacy .xls files need engine='xlrd'.
`pd.read_excel('report.xlsx', sheet_name=None)`	Passing None for sheet_name reads every worksheet in the workbook, returning a dictionary where each key is a sheet name and each value is that sheet's DataFrame.

5.3 Reading JSON, SQL and Parquet

```
df_json = pd.read_json('data.json', orient='records')
df_sql = pd.read_sql('SELECT * FROM employees WHERE active = 1', con=engine)
df_parquet = pd.read_parquet('data.parquet', engine='pyarrow')
df_html = pd.read_html('https://example.com/table-page.html')[0]
```

Code Line	Explanation
`orient='records'`	Tells pandas the JSON is structured as a list of row-like dictionaries (the most common API response shape); other options include 'columns', 'index' and 'split' for differently shaped JSON.
`pd.read_sql(query, con=engine)`	Executes a SQL query string against an open SQLAlchemy engine or database connection and loads the result set directly into a DataFrame.
`pd.read_parquet('data.parquet', engine='pyarrow')`	Reads a Parquet file, a compressed columnar format, using the pyarrow engine; Parquet preserves data types exactly and is much faster to read than CSV for large files.
`pd.read_html(url)[0]`	Scans a web page for all HTML <table> elements and returns a list of DataFrames, one per table found; [0] selects the first table on the page.

5.4 Chunked Reading for Large Files

```
chunks = []
for chunk in pd.read_csv('huge_file.csv', chunksize=100000):
    processed = chunk[chunk['amount'] > 0]
    chunks.append(processed)
df = pd.concat(chunks, ignore_index=True)
```

Code Line	Explanation
`pd.read_csv('huge_file.csv', chunksize=100000)`	Instead of loading the whole file at once, returns an iterator that yields the file 100,000 rows at a time, keeping memory usage bounded regardless of total file size.
`for chunk in ...:`	Loops over each chunk in turn, treating every chunk as its own small DataFrame.
`chunk[chunk['amount'] > 0]`	Filters each chunk down to only the rows needed, before accumulating results, so the final concatenation stays as small as possible.
`pd.concat(chunks, ignore_index=True)`	Stitches the list of filtered chunk DataFrames back together into one final DataFrame; ignore_index=True discards each chunk's original row numbers and generates a fresh, continuous index.

6. Initial Data Inspection

6.1 Shape and Preview

```
df.shape
df.head(10)
df.tail(5)
df.sample(5, random_state=1)
```

Code Line	Explanation
`df.shape`	Returns (rows, columns), the size of the DataFrame.
`df.head(10)`	Displays the first 10 rows; the default with no argument is 5.
`df.tail(5)`	Displays the last 5 rows, useful for spotting trailing footer or summary rows accidentally included in a file.
`df.sample(5, random_state=1)`	Displays 5 randomly chosen rows; random_state fixes the random seed so the same sample is returned every time the code runs, making the output reproducible.

6.2 Structural Summary

```
df.info()
df.info(memory_usage='deep')
```

Code Line	Explanation
`df.info()`	Prints column names, the count of non-null values per column, each column's data type, and total memory usage; the fastest single check for missing values and wrong types.
`memory_usage='deep'`	Requests a more accurate memory calculation that inspects the actual contents of object (text) columns rather than estimating, since text columns can otherwise be significantly under-reported.

6.3 Descriptive Statistics

```
df.describe()
df.describe(include='all')
df.describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9])
```

Code Line	Explanation
`df.describe()`	Computes count, mean, std, min, 25/50/75 percentiles and max for every numeric column.
`include='all'`	Extends the summary to non-numeric columns too, adding unique count, top (most frequent) value and its frequency.
`percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]`	Overrides the default quartiles with a custom list of percentiles to display, here also including the 10th and 90th percentiles.

6.4 Unique Values and Value Counts

```
df['department'].unique()
```

```
df['department'].nunique()
df['department'].value_counts()
df.nunique()
```

Code Line	Explanation
`df['department'].unique()`	Returns an array of every distinct value found in the column, in order of first appearance.
`df['department'].nunique()`	Returns the count of distinct values.
`df['department'].value_counts()`	Counts how many times each distinct value appears, sorted from most to least frequent.
`df.nunique()`	Applied to the whole DataFrame, returns the count of distinct values for every column at once, useful for quickly spotting which columns are near-constant or highly unique (like an ID column).

7. Selecting and Indexing Data

7.1 Selecting Columns

```
df['salary']
df.salary
df[['name', 'salary']]
```

Code Line	Explanation
`df['salary']`	Selects a single column using square-bracket syntax, returning it as a Series. This works for any column name, including ones with spaces.
`df.salary`	Selects the same column using attribute (dot) access; this is more convenient to type but fails for column names with spaces or that clash with existing DataFrame method names (like df.count).
`df[['name', 'salary']]`	Selects multiple columns at once using a list of column names inside the square brackets; the double brackets are required, the outer pair for indexing and the inner pair to define the list of names. This returns a DataFrame, not a Series, even if only one column is listed.

7.2 loc: Label-Based Selection

```
df.loc[0]
df.loc[0:4]
df.loc[0:4, 'name']
df.loc[0:4, ['name', 'salary']]
df.loc[df['salary'] > 80000, ['name', 'department']]
```

Code Line	Explanation
`df.loc[0]`	Selects the row labeled 0 (not necessarily the first physical row, if the index has been changed), returned as a Series.
`df.loc[0:4]`	Selects rows labeled 0 through 4 inclusive; unlike Python's normal slicing, .loc includes the endpoint (4) in the result.
`df.loc[0:4, 'name']`	Restricts the selection to only the 'name' column within that row range, using a comma to separate the row selector from the column selector.
`df.loc[0:4, ['name', 'salary']]`	Selects both a row range and a specific list of columns simultaneously.
`df.loc[df['salary'] > 80000, ['name', 'department']]`	Combines a boolean condition for row selection (only rows where salary exceeds 80000) with a specific column list, one of the most common patterns in pandas: filter and select in a single call.

7.3 iloc: Position-Based Selection

```
df.iloc[0]
df.iloc[0:5]
df.iloc[0:5, 0:2]
df.iloc[-1]
df.iloc[[0, 2, 4]]
```

Code Line	Explanation
`df.iloc[0]`	Selects the row at integer position 0 (the very first physical row), regardless of what its label is.
`df.iloc[0:5]`	Selects rows at positions 0 through 4; like standard Python slicing, the endpoint (5) is excluded, unlike .loc.
`df.iloc[0:5, 0:2]`	Selects a block of rows and columns purely by position: the first 5 rows and the first 2 columns.
`df.iloc[-1]`	Selects the last row, using Python's negative indexing convention.
`df.iloc[[0, 2, 4]]`	Selects specific, non-contiguous row positions by passing a list of integer positions rather than a slice.

7.4 at and iat: Fast Scalar Access

```
df.at[3, 'salary']
df.iat[3, 2]
```

df.at and df.iat are optimized for accessing or setting a single scalar value, label-based and position-based respectively. They are faster than .loc or .iloc for this specific case because they skip the extra overhead involved in supporting more general slicing and array-shaped selections.

7.5 Setting Values With loc

```
df.loc[df['department'] == 'Sales', 'bonus_eligible'] = True
df.loc[5, 'salary'] = 95000
```

Code Line	Explanation
`df.loc[df['department'] == 'Sales', 'bonus_eligible'] = True`	Uses a boolean condition to locate every row where department equals 'Sales', then assigns True to the bonus_eligible column only for those matched rows; other rows are left untouched.
`df.loc[5, 'salary'] = 95000`	Directly overwrites the salary value in the row labeled 5 with a new number.

Warning: Always use .loc (or .iloc) to assign values, rather than chained indexing like df[df['x'] > 0]['y'] = 1. Chained indexing can silently fail to modify the original DataFrame and triggers pandas' well-known SettingWithCopyWarning.

8. Filtering and Boolean Indexing

8.1 Basic Boolean Filtering

```
df[df['salary'] > 70000]
df[df['department'] == 'Engineering']
df[df['salary'].between(50000, 90000)]
```

Code Line	Explanation
`df['salary'] > 70000`	Evaluated on its own, this produces a Series of True/False values, one per row, indicating whether that row's salary exceeds 70000.
`df[ ... ]`	Wrapping that boolean Series in square brackets around df filters the DataFrame, keeping only the rows where the condition is True.
`.between(50000, 90000)`	A convenience method equivalent to writing (df['salary'] >= 50000) & (df['salary'] <= 90000), but more concise and readable; inclusive of both endpoints by default.

8.2 Combining Multiple Conditions

```
df[(df['salary'] > 70000) & (df['department'] == 'Engineering')]
df[(df['department'] == 'Sales') | (df['department'] == 'Marketing')]
df[~(df['department'] == 'Sales')]
```

Code Line	Explanation
`&`	The element-wise logical AND operator, used instead of Python's plain 'and' keyword because pandas needs to evaluate the comparison position by position across the whole column at once.
`|`	The element-wise logical OR operator, used instead of Python's plain 'or' keyword for the same reason.
`~`	The element-wise logical NOT operator, which inverts a boolean condition; here it excludes the Sales department instead of including it.
`Parentheses around each condition`	Required because Python evaluates & and | with higher precedence than comparison operators like > or ==, so omitting the parentheses causes a confusing error or wrong result.

8.3 isin() for Membership Checks

```
df[df['department'].isin(['Sales', 'Marketing', 'Support'])]
df[~df['department'].isin(['Sales', 'Marketing', 'Support'])]
```

Code Line	Explanation
`.isin(['Sales', 'Marketing', 'Support'])`	Checks each value in the department column against the given list, returning True for any row whose value matches one of the three listed departments; a cleaner alternative to chaining several == conditions with |.

8.4 query() for Readable Filters

```
df.query('salary > 70000 and department == "Engineering"')
min_salary = 60000
df.query('salary > @min_salary')
```

Code Line	Explanation
`df.query('salary > 70000 and department == "Engineering"')`	Filters the DataFrame using a condition written as a plain text string, which can be more readable than repeated bracket-and-ampersand syntax, especially for filters with several conditions.
`@min_salary`	Inside a query string, the @ symbol references a Python variable from the surrounding code (here min_salary), letting the filter threshold be defined dynamically instead of hardcoded into the string.

9. Handling Missing Data

9.1 Detecting Missing Values

```
df.isnull().sum()
df.isnull().mean() * 100
df[df['salary'].isnull()]
df.notnull().sum()
```

Code Line	Explanation
`df.isnull().sum()`	Counts the number of missing (NaN) values in each column.
`df.isnull().mean() * 100`	The mean of a boolean column is the fraction of True values; multiplying by 100 converts that fraction of missing values into a percentage per column.
`df[df['salary'].isnull()]`	Filters the DataFrame down to only the rows where the salary column is missing, useful for inspecting exactly which records are incomplete.
`df.notnull().sum()`	The logical opposite of isnull(); counts how many non-missing values exist in each column.

9.2 Dropping Missing Values

```
df.dropna()
df.dropna(subset=['salary', 'hire_date'])
df.dropna(how='all')
df.dropna(axis=1, thresh=int(0.8 * len(df)))
```

Code Line	Explanation
`df.dropna()`	Removes any row containing at least one missing value in any column.
`subset=['salary', 'hire_date']`	Restricts the check to only these named columns, ignoring missing values elsewhere.
`how='all'`	Only drops a row if every single value in it is missing, rather than just one; much less aggressive than the default 'any' behavior.
`axis=1, thresh=int(0.8 * len(df))`	Switches to dropping columns instead of rows (axis=1), and keeps a column only if it has at least 80 percent of its values present (thresh sets the minimum number of non-null values required).

9.3 Filling Missing Values

```
df['salary'] = df['salary'].fillna(df['salary'].median())
df['department'] = df['department'].fillna('Unassigned')
df['salary'] = df.groupby('department')['salary'].transform(lambda x:
x.fillna(x.median()))
df = df.fillna(method='ffill')
df = df.interpolate(method='linear')
```

Code Line	Explanation
`fillna(df['salary'].median())`	Replaces missing salary values with the overall median salary, a robust central value less affected by outliers than the mean.
`fillna('Unassigned')`	Replaces missing text values with a fixed placeholder string.
`groupby('department')['salary'].transform(lambda x: x.fillna(x.median()))`	A more precise imputation: groups the data by department first, then fills each department's missing salaries with that specific department's own median, rather than a single global median; .transform() returns a result aligned back to the original row index so it can be assigned straight back into the column.
`fillna(method='ffill')`	Forward-fills every missing value across the whole DataFrame with the last valid value seen above it, appropriate for ordered or time series data.
`df.interpolate(method='linear')`	Estimates missing numeric values by drawing a straight line between the nearest valid values before and after the gap, a common technique for filling gaps in continuous measurements.

9.4 Flagging Missingness Before Filling

```
df['salary_missing'] = df['salary'].isna()
```

Creating a boolean flag column before imputation preserves the information that a value was originally missing, which can itself carry predictive signal (for example, employees with no reported salary might belong to a specific data collection gap or employment type).

10. Handling Duplicate Records

10.1 Detecting Duplicates

```
df.duplicated().sum()
df[df.duplicated(keep=False)]
df.duplicated(subset=['name', 'hire_date']).sum()
```

Code Line	Explanation
`df.duplicated()`	Returns a boolean Series marking True for every row that is an exact repeat of an earlier row; by default the first occurrence is marked False (not a duplicate) and later copies are marked True.
`keep=False`	Marks every copy of a duplicated row as True, including the first occurrence, which is useful when the goal is to inspect all instances of the duplication rather than just the extras.
`subset=['name', 'hire_date']`	Limits the duplicate comparison to only these two columns, so a row counts as a duplicate if this specific combination repeats, even if other columns differ.

10.2 Removing Duplicates

```
df = df.drop_duplicates()
df = df.drop_duplicates(subset=['employee_id'], keep='last')
```

Code Line	Explanation
`df.drop_duplicates()`	Removes rows that are exact duplicates of an earlier row, keeping the first occurrence by default.
`keep='last'`	Keeps the last occurrence of each duplicate group instead of the first, useful when later rows represent more recent or more accurate data.

10.3 Finding Near-Duplicates

```
df['name_clean'] = df['name'].str.strip().str.lower()
df[df.duplicated(subset=['name_clean'], keep=False)].sort_values('name_clean')
```

Exact duplicate detection misses records that differ only by capitalization or stray whitespace, such as 'Ada Lovelace' versus 'ada lovelace '. Standardizing a column first (trim and lowercase) before checking for duplicates catches these near-duplicates that would otherwise be missed.

11. Data Type Conversion

11.1 Common Conversions

```
df['hire_date'] = pd.to_datetime(df['hire_date'], format='%Y-%m-%d',
errors='coerce')
df['salary'] = pd.to_numeric(df['salary'], errors='coerce')
df['employee_id'] = df['employee_id'].astype(str)
df['is_manager'] = df['is_manager'].astype(bool)
df['department'] = df['department'].astype('category')
```

Code Line	Explanation
`pd.to_datetime(..., format='%Y-%m-%d', errors='coerce')`	Parses text into datetime objects; supplying an explicit format string speeds up parsing significantly and avoids ambiguity (for example, is '01/02/2024' January 2nd or February 1st); errors='coerce' converts any unparseable value to NaT (Not a Time) instead of raising an exception.
`pd.to_numeric(df['salary'], errors='coerce')`	Converts text to a numeric type, turning any value that cannot be parsed (like the text 'confidential') into NaN rather than stopping execution.
`.astype(str)`	Forces a column to be stored as text; commonly used for ID columns so they are never accidentally treated as numbers (which would break leading zeros or attempt arithmetic on them).
`.astype(bool)`	Converts a column to True/False boolean values.
`.astype('category')`	Converts a column with a limited set of repeating text values into pandas' category dtype, which stores each unique value once internally, using substantially less memory and speeding up groupby operations.

11.2 Checking and Converting All Columns at Once

```
df.dtypes
numeric_cols = df.select_dtypes(include='number').columns
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors='coerce')
```

Code Line	Explanation
`df.select_dtypes(include='number')`	Returns a DataFrame containing only the columns whose dtype is numeric (int or float), useful for isolating numeric columns without listing their names manually.
`.columns`	Extracts just the column names from that filtered DataFrame.
`df[numeric_cols].apply(pd.to_numeric, errors='coerce')`	Applies the pd.to_numeric conversion function to every column in that list at once, cleaning any stray text characters across all numeric columns in a single line.

11.3 Downcasting Numeric Types to Save Memory

```
df['age'] = pd.to_numeric(df['age'], downcast='integer')
df['salary'] = pd.to_numeric(df['salary'], downcast='float')
```

downcast='integer' finds the smallest integer subtype (such as int8, int16 or int32) that can safely represent the column's actual range of values, instead of using the default, wider int64. The same idea applies to downcast='float'. On large datasets this can substantially reduce memory usage with no loss of information.

12. String Operations

12.1 The .str Accessor

Pandas provides a special .str accessor on text (object or string dtype) Series, unlocking vectorized string methods that mirror Python's built-in string methods but apply to an entire column at once, without writing an explicit loop.

```
df['name'] = df['name'].str.strip()
df['name'] = df['name'].str.title()
df['email_domain'] = df['email'].str.split('@').str[1]
df['is_gmail'] = df['email'].str.contains('gmail.com', case=False, na=False)
df['name_length'] = df['name'].str.len()
```

Code Line	Explanation
`.str.strip()`	Removes leading and trailing whitespace from every value in the column.
`.str.title()`	Capitalizes the first letter of every word in each string, for example 'ada lovelace' becomes 'Ada Lovelace'.
`.str.split('@')`	Splits each string on the @ character, returning a Series where every element is a list of the resulting pieces.
`.str[1]`	Chained after split(), this indexes into each resulting list to grab the second piece (position 1), effectively extracting the domain portion of an email address.
`.str.contains('gmail.com', case=False, na=False)`	Checks whether each string contains the given substring, returning True/False; case=False makes the match case-insensitive, and na=False ensures missing values produce False instead of raising an error or propagating NaN.
`.str.len()`	Returns the character length of each string in the column.

12.2 Replacing and Extracting Text

```
df['phone'] = df['phone'].str.replace(r'[^0-9]', '', regex=True)
df['area_code'] = df['phone'].str.extract(r'^(\d{3})')
df['name_parts'] = df['name'].str.split(' ', n=1, expand=True)
```

Code Line	Explanation
`.str.replace(r'[^0-9]', '', regex=True)`	Uses a regular expression to remove every character that is not a digit, cleaning a messy phone number field down to digits only; regex=True tells pandas to interpret the pattern as a regular expression rather than a literal string.
`.str.extract(r'^(\d{3})')`	Applies a regular expression with a capture group (the parentheses) to pull out just the first 3 digits from the start of each string, here extracting an area code.
`.str.split(' ', n=1, expand=True)`	Splits each name on the first space only (n=1 limits it to one split); expand=True returns the result as a full DataFrame with one column per split piece, instead of a Series of lists.

12.3 Common .str Methods Reference

Code Line	Explanation
`.str.lower() / .str.upper()`	Convert to lowercase or uppercase.
`.str.startswith(x) / .str.endswith(x)`	Check whether each string starts or ends with the given substring.
`.str.zfill(n)`	Pads each string on the left with zeros until it reaches length n, useful for fixing ID codes that lost leading zeros.
`.str.slice(start, stop)`	Extracts a substring by character position, similar to Python's slicing syntax.
`.str.cat(sep=', ')`	Concatenates all the strings in the column into a single combined string, joined by the given separator.
`.str.pad(width, side='left')`	Pads each string with spaces (or a custom character) to reach a fixed width.
`.str.get_dummies(sep='|')`	Splits each string on the separator and one-hot encodes every distinct token found, useful for multi-value tag fields like 'python|sql|excel'.

13. Applying Functions and Vectorization

13.1 Vectorized Operations (Always Prefer These First)

```
df['bonus'] = df['salary'] * 0.10
df['total_comp'] = df['salary'] + df['bonus']
df['salary_k'] = df['salary'] / 1000
```

These lines look like simple arithmetic, but each one is a vectorized operation: pandas applies the multiplication, addition or division to every row simultaneously using fast, compiled code, without Python ever looping through the rows one at a time. Vectorized operations like these should always be the first choice; they are typically 10 to 100 times faster than an equivalent loop or .apply() call, because the row-by-row work happens in optimized C code instead of the slower Python interpreter.

13.2 apply() on a Series

```
df['salary_band'] = df['salary'].apply(lambda x: 'High' if x > 90000 else
'Standard')

def classify(salary):
    if salary > 90000:
        return 'High'
    elif salary > 60000:
        return 'Standard'
    return 'Entry'

df['salary_band'] = df['salary'].apply(classify)
```

Code Line	Explanation
`.apply(lambda x: 'High' if x > 90000 else 'Standard')`	Calls the given function once for every value in the Series, passing that single value in as x, and collects the returned results into a new Series; a lambda is a small, unnamed inline function, useful for short one-off logic.
`def classify(salary): ...`	Defines a regular, named Python function with more complex branching logic than would fit comfortably in a lambda.
`.apply(classify)`	Passes the named function instead of a lambda; pandas calls classify() once per value in the salary column, exactly as it would with a lambda.

13.3 apply() on a DataFrame (Row-Wise)

```
df['summary'] = df.apply(lambda row: f"{row['name']} earns {row['salary']}", axis=1)
```

Code Line	Explanation
`df.apply(lambda row: ..., axis=1)`	When called on a whole DataFrame with axis=1, the function is called once per row, and the entire row is passed in as a Series (accessed here as row), allowing the function to reference multiple columns from the same row at once.

Code Line	Explanation
`axis=1`	Specifies that the function should be applied across each row (moving down the columns is axis=0, the default; moving across the columns within a row is axis=1).

Performance Tip: Row-wise apply(axis=1) is convenient but much slower than a vectorized alternative, because pandas still loops through rows in Python internally. Whenever the same logic can be expressed with column-wise arithmetic, boolean masks, or np.where/np.select, prefer that instead.

13.4 np.where and np.select for Fast Conditional Logic

```
df['salary_band'] = np.where(df['salary'] > 90000, 'High', 'Standard')

conditions = [df['salary'] > 90000, df['salary'] > 60000]
choices = ['High', 'Standard']
df['salary_band'] = np.select(conditions, choices, default='Entry')
```

Code Line	Explanation
`np.where(condition, value_if_true, value_if_false)`	A fully vectorized ternary operation: evaluates the boolean condition for every row at once and picks one of the two given values accordingly, avoiding the row-by-row overhead of .apply().
`np.select(conditions, choices, default='Entry')`	Extends np.where to handle multiple conditions in priority order; conditions is a list of boolean Series and choices is the matching list of results, evaluated top to bottom, with default used when none of the conditions match.

13.5 map() for Value Substitution

```
dept_codes = {'Engineering': 'ENG', 'Sales': 'SAL', 'Marketing': 'MKT'}
df['dept_code'] = df['department'].map(dept_codes)
```

.map() applied to a Series looks up each value in the given dictionary (or calls the given function) and replaces it with the corresponding result; values not found in the dictionary become NaN. It is a fast, purpose-built tool for exactly this kind of one-to-one value substitution, and is generally faster than an equivalent .apply() with an if/elif chain.

13.6 applymap / DataFrame.map for Element-Wise Operations

```
df[['q1', 'q2', 'q3', 'q4']] = df[['q1', 'q2', 'q3', 'q4']].map(lambda x: round(x,
2))
```

This applies a function to every individual element of the selected columns, independent of row or column context; useful for formatting or transforming every cell in a block of numeric columns at once. In pandas versions before 2.1 this was written as .applymap() instead of .map() on a DataFrame.

14. Sorting and Ranking

14.1 Sorting by Values

```
df.sort_values('salary')
df.sort_values('salary', ascending=False)
df.sort_values(['department', 'salary'], ascending=[True, False])
df.sort_values('salary', na_position='first')
```

Code Line	Explanation
`df.sort_values('salary')`	Sorts the whole DataFrame by the salary column in ascending order (smallest first) by default.
`ascending=False`	Reverses the sort order to descending (largest first).
`sort_values(['department', 'salary'], ascending=[True, False])`	Sorts by multiple columns in sequence: first groups rows by department alphabetically, then within each department orders rows by salary from highest to lowest; the ascending list must match the order and length of the sort column list.
`na_position='first'`	Controls where missing values are placed in the sorted result; by default they are pushed to the end ('last').

14.2 Sorting by Index

```
df.sort_index()
df.sort_index(ascending=False)
df.sort_index(axis=1)
```

Code Line	Explanation
`df.sort_index()`	Sorts the DataFrame by its row labels rather than by any column's values, restoring order after operations (like a groupby) that can leave rows in a non-obvious order.
`sort_index(axis=1)`	Sorts the columns alphabetically by name instead of sorting the rows.

14.3 Ranking Values

```
df['salary_rank'] = df['salary'].rank(ascending=False, method='dense')
```

Code Line	Explanation
`.rank(ascending=False)`	Assigns a numeric rank to each value; ascending=False means the highest salary gets rank 1.
`method='dense'`	Controls how tied values are ranked; 'dense' gives tied values the same rank and does not skip the next rank number afterward (1, 1, 2 instead of 1, 1, 3), unlike the default 'average' method which splits ties.

14.4 nlargest and nsmallest

```
df.nlargest(5, 'salary')
df.nsmallest(5, 'salary')
```

These are optimized shortcuts equivalent to sorting the entire DataFrame and then taking a head() or tail() slice, but computed more efficiently since pandas does not need to fully sort every row, only identify the top or bottom n.

15. Grouping and Aggregation (GroupBy)

15.1 The Split-Apply-Combine Pattern

groupby() implements a three-stage pattern: first the data is split into groups based on one or more keys, then a function is applied independently within each group (like a sum, mean, or custom calculation), and finally the results are combined back into a single output structure. This pattern underlies almost every aggregation or per-category calculation in pandas.

15.2 Basic Grouping and Aggregation

```
df.groupby('department')['salary'].mean()
df.groupby('department')['salary'].sum()
df.groupby('department').size()
df.groupby('department')['salary'].agg(['mean', 'median', 'std', 'count'])
```

Code Line	Explanation
`df.groupby('department')`	Splits the DataFrame into a separate group for every unique value in the department column; nothing is computed yet at this point, it just prepares the grouping.
`['salary'].mean()`	Selects the salary column within each group and computes its average, returning one mean value per department.
`df.groupby('department').size()`	Counts the number of rows in each group, including rows with missing values in other columns (unlike .count(), which counts only non-null entries per column).
`.agg(['mean', 'median', 'std', 'count'])`	Computes several summary statistics on the grouped salary column at once, returning a small table with one row per department and one column per statistic.

15.3 Grouping by Multiple Keys

```
df.groupby(['department', 'is_manager'])['salary'].mean()
df.groupby(['department', 'is_manager'])['salary'].mean().reset_index()
```

Code Line	Explanation
`groupby(['department', 'is_manager'])`	Forms a separate group for every unique combination of department and is_manager, producing a result with a two-level (hierarchical) index.
`.reset_index()`	Converts the grouped, multi-level-indexed result back into a flat, ordinary DataFrame with department and is_manager restored as regular columns, which is usually easier to work with downstream.

15.4 Custom Aggregations With agg()

```
df.groupby('department').agg(
    avg_salary=('salary', 'mean'),
    max_salary=('salary', 'max'),
    headcount=('employee_id', 'nunique')
```

```
)
```

Code Line	Explanation
`avg_salary=('salary', 'mean')`	This is 'named aggregation' syntax: the keyword avg_salary becomes the resulting output column's name, and the tuple ('salary', 'mean') specifies which source column to aggregate and which function to apply to it.
`headcount=('employee_id', 'nunique')`	Counts the number of distinct employee_id values within each department group, useful when a department might have duplicate rows per employee and a plain row count would overcount.

15.5 transform() vs apply() in GroupBy

```
df['dept_avg_salary'] = df.groupby('department')['salary'].transform('mean')
df['salary_vs_dept_avg'] = df['salary'] - df['dept_avg_salary']
```

Code Line	Explanation
`.transform('mean')`	Computes the mean within each group, exactly like .agg('mean'), but instead of collapsing the result to one row per group, it broadcasts that group's mean back out to every original row that belongs to that group, so the result has the same length as the original DataFrame and can be assigned directly as a new column.
`df['salary'] - df['dept_avg_salary']`	Because both columns now share the same row-aligned length, this line can directly compute how far each individual's salary deviates from their own department's average, entirely through vectorized subtraction.

15.6 Filtering Groups

```
df.groupby('department').filter(lambda g: len(g) >= 5)
```

.filter() on a groupby object keeps or drops entire groups based on a condition evaluated on each group as a whole, here keeping only departments that have at least 5 employees; this is different from filtering individual rows, since a whole group either passes or is dropped together.

16. Merging, Joining and Concatenating

16.1 merge(): SQL-Style Joins

```
employees = pd.DataFrame({'emp_id': [1, 2, 3], 'name': ['Ada', 'Grace', 'Alan']})
salaries = pd.DataFrame({'emp_id': [1, 2, 4], 'salary': [95000, 88000, 71000]})

pd.merge(employees, salaries, on='emp_id', how='inner')
pd.merge(employees, salaries, on='emp_id', how='left')
pd.merge(employees, salaries, on='emp_id', how='outer', indicator=True)
```

Code Line	Explanation
`on='emp_id'`	The shared key column used to match rows between the two DataFrames; both tables must contain a column with this name (or left_on/right_on can specify different names in each table).
`how='inner'`	Keeps only rows where the key exists in both DataFrames; emp_id 3 (only in employees) and emp_id 4 (only in salaries) are dropped.
`how='left'`	Keeps every row from the left DataFrame (employees) regardless of a match, filling with NaN where no matching salary exists (emp_id 3).
`how='outer'`	Keeps every row from both DataFrames, filling with NaN wherever a match is missing on either side.
`indicator=True`	Adds an extra column named _merge to the result, labeling each row as 'left_only', 'right_only' or 'both', which is useful for auditing exactly how the join resolved each row.

16.2 Joining on the Index

```
employees_indexed = employees.set_index('emp_id')
salaries_indexed = salaries.set_index('emp_id')
employees_indexed.join(salaries_indexed, how='left')
```

Code Line	Explanation
`.set_index('emp_id')`	Moves the emp_id column out of the regular columns and makes it the DataFrame's row index instead.
`.join(salaries_indexed, how='left')`	Combines two DataFrames using their row index as the matching key rather than a named column; conceptually similar to merge(), but defaults to matching on the index and is convenient when both tables are already indexed the same way.

16.3 concat(): Stacking DataFrames

```
q1 = pd.DataFrame({'month': ['Jan'], 'sales': [1000]})
q2 = pd.DataFrame({'month': ['Apr'], 'sales': [1200]})
pd.concat([q1, q2], ignore_index=True)
pd.concat([employees, salaries], axis=1)
```

Code Line	Explanation
`pd.concat([q1, q2], ignore_index=True)`	Stacks the two DataFrames on top of each other (the default axis=0),

Code Line	Explanation
	combining their rows into one longer table; ignore_index=True discards each original index and generates a fresh, continuous one instead of keeping possibly duplicated old row labels.
`pd.concat([employees, salaries], axis=1)`	With axis=1, concatenates the DataFrames side by side as new columns instead of stacking rows, aligning them by their existing index; this only makes sense when both DataFrames already share a compatible index.

merge() vs concat(): merge() is for combining tables based on matching key values (like a SQL JOIN). concat() is for simply stacking tables that already share the same structure, such as appending a new month's data onto an existing table.

17. Reshaping Data (Pivot, Melt, Stack/Unstack)

17.1 pivot_table(): Long to Wide

```
pd.pivot_table(
    df,
    values='sales',
    index='region',
    columns='quarter',
    aggfunc='sum',
    fill_value=0,
    margins=True
)
```

Code Line	Explanation
`values='sales'`	The column whose values will fill the cells of the resulting pivot table.
`index='region'`	The column whose unique values become the row labels of the new table.
`columns='quarter'`	The column whose unique values become the new column headers, spreading what was one long column out into several wide columns.
`aggfunc='sum'`	The function used to combine values whenever more than one row shares the same region and quarter combination; here they are summed together.
`fill_value=0`	Replaces any resulting empty cell (a region/quarter combination with no matching data) with 0 instead of leaving it as NaN.
`margins=True`	Adds an extra 'All' row and column showing the grand total across each row and column, similar to a subtotal row in a spreadsheet pivot table.

17.2 melt(): Wide to Long

```
wide = pd.DataFrame({'region': ['East', 'West'], 'Q1': [100, 150], 'Q2': [120,
160]})
pd.melt(
    wide,
    id_vars='region',
    value_vars=['Q1', 'Q2'],
    var_name='quarter',
    value_name='sales'
)
```

Code Line	Explanation
`id_vars='region'`	The column(s) that should stay fixed and simply repeat for each melted row; here every region will appear twice, once per quarter.
`value_vars=['Q1', 'Q2']`	The columns that should be unpivoted, taking their column names and values and stacking them into two new columns instead of staying spread out horizontally.
`var_name='quarter'`	The name given to the new column that will hold the former column

Code Line	Explanation
	headers ('Q1', 'Q2').
`value_name='sales'`	The name given to the new column that will hold the actual values that used to live under each quarter column.

melt() is the inverse of pivot_table(): it converts a wide table (one column per category) into a long, tidy table (one row per observation), which is often the format required by plotting libraries and statistical models.

17.3 stack() and unstack()

```
stacked = wide.set_index('region').stack()
unstacked = stacked.unstack()
```

Code Line	Explanation
`.stack()`	Pivots the innermost column level into the innermost row index level, converting a wide DataFrame into a long Series with a two-level (region, quarter) MultiIndex.
`.unstack()`	The inverse operation, pivoting the innermost row index level back out into columns, restoring the original wide shape.

17.4 wide_to_long() and crosstab()

```
pd.crosstab(df['department'], df['is_manager'])
pd.crosstab(df['department'], df['is_manager'], normalize='index')
```

Code Line	Explanation
`pd.crosstab(a, b)`	Builds a frequency table counting how many rows fall into every combination of the two given columns; similar to a simple pivot_table with aggfunc='count' but more concise for this specific use case.
`normalize='index'`	Converts each row's raw counts into proportions of that row's total, useful for comparing composition across categories with very different total sizes.

18. MultiIndex (Hierarchical Indexing)

18.1 What a MultiIndex Is

A MultiIndex, also called a hierarchical index, allows a DataFrame or Series to have more than one level of row labels at once, for example region and quarter together. It naturally arises from operations like groupby() with multiple keys, pivot_table(), or set_index() with a list of columns, and it makes it possible to represent higher-dimensional data within pandas' fundamentally two-dimensional structure.

18.2 Creating a MultiIndex

```
df_multi = df.set_index(['department', 'employee_id'])
df_multi.index
df_multi.index.get_level_values('department')
```

Code Line	Explanation
`set_index(['department', 'employee_id'])`	Passing a list of column names instead of a single name creates a two-level MultiIndex, with department as the outer level and employee_id as the inner level.
`.index.get_level_values('department')`	Extracts just the values from one specific named level of the MultiIndex, returned as a plain Index, useful for inspecting or filtering by a single level.

18.3 Selecting Data From a MultiIndex

```
df_multi.loc['Engineering']
df_multi.loc[('Engineering', 101)]
df_multi.xs('Engineering', level='department')
```

Code Line	Explanation
`df_multi.loc['Engineering']`	Selects every row where the outer index level equals 'Engineering', dropping that level from the result.
`df_multi.loc[('Engineering', 101)]`	Selects a specific row by supplying a tuple with a value for every level of the MultiIndex, here department 'Engineering' and employee_id 101 together.
`df_multi.xs('Engineering', level='department')`	The cross-section method offers a more explicit way to select by a named level regardless of its position, which is especially useful with more than two levels.

18.4 Resetting and Renaming Index Levels

```
df_multi.reset_index()
df_multi.rename_axis(['dept', 'emp_id'])
```

Code Line	Explanation
`.reset_index()`	Converts every level of the MultiIndex back into ordinary columns and replaces it with a fresh default integer index, flattening the hierarchy.

Code Line	Explanation
`.rename_axis(['dept', 'emp_id'])`	Renames the levels of the index itself (the level names shown above the index), without changing any of the underlying data.

19. Time Series Handling

19.1 Creating and Parsing Datetime Data

```
df['order_date'] = pd.to_datetime(df['order_date'])
date_range = pd.date_range(start='2026-01-01', end='2026-01-10', freq='D')
df = df.set_index('order_date')
```

Code Line	Explanation
`pd.to_datetime(df['order_date'])`	Converts a text column into pandas Timestamp objects, unlocking every date-based operation used below.
`pd.date_range(start=..., end=..., freq='D')`	Generates an evenly spaced sequence of dates between the start and end, here one per day ('D'); other common frequencies include 'W' (weekly), 'M' (month end), 'MS' (month start), 'Q' (quarter end), and 'H' (hourly).
`df.set_index('order_date')`	Makes the datetime column the DataFrame's row index, which is required to use pandas' time-series-specific tools like resample() and many convenient slicing patterns.

19.2 Extracting Date Components

```
df['year'] = df.index.year
df['month'] = df.index.month
df['day_name'] = df.index.day_name()
df['quarter'] = df.index.quarter
df['is_month_end'] = df.index.is_month_end
```

Code Line	Explanation
`df.index.year / .month / .quarter`	Extract the numeric year, month or quarter directly from a datetime index.
`df.index.day_name()`	Returns the full weekday name as text, such as 'Monday'.
`df.index.is_month_end`	Returns True/False for whether each date is the final calendar day of its month, useful for flagging month-end reporting rows.

19.3 Date-Based Slicing

```
df.loc['2026-01']
df.loc['2026-01-01':'2026-01-15']
df.loc['2026']
```

When the index is a proper DatetimeIndex, pandas allows partial-string slicing: writing '2026-01' selects every row within January 2026 without needing to compute exact start and end timestamps manually. A full date range can also be sliced directly, and .loc slicing on a datetime index is inclusive of both endpoints.

19.4 Resampling (Changing the Time Frequency)

```
df['sales'].resample('D').sum()
df['sales'].resample('W').mean()
df['sales'].resample('M').agg(['sum', 'mean', 'count'])
```

Code Line	Explanation
`.resample('D')`	Groups the data into fixed daily time buckets, similar in spirit to groupby() but specifically for time-based intervals; 'D' means daily, 'W' weekly, 'M' month-end, and so on.
`.resample('D').sum()`	Within each daily bucket, sums all the sales values that fall in that day, collapsing possibly many rows per day into one summary row.
`.resample('M').agg(['sum', 'mean', 'count'])`	Applies several aggregation functions at once to each monthly bucket, producing one row per month with multiple summary statistics.

19.5 Shifting and Differencing

```
df['sales_prev_day'] = df['sales'].shift(1)
df['sales_change'] = df['sales'].diff()
df['sales_pct_change'] = df['sales'].pct_change()
```

Code Line	Explanation
`.shift(1)`	Moves every value in the column forward by one position, so each row now shows the previous row's value; commonly used to compare a value against the prior period.
`.diff()`	Calculates the difference between each value and the one immediately before it, equivalent to df['sales'] - df['sales'].shift(1).
`.pct_change()`	Calculates the percentage change from the previous value to the current one, useful for measuring growth rate period over period.

19.6 Time Zones

```
df.index = df.index.tz_localize('UTC')
df.index = df.index.tz_convert('Africa/Lagos')
```

Code Line	Explanation
`.tz_localize('UTC')`	Attaches a time zone label to datetime values that currently have none (are timezone-naive), declaring that they should be interpreted as UTC without changing the underlying clock time.
`.tz_convert('Africa/Lagos')`	Converts already timezone-aware datetimes from their current zone into a different target zone, adjusting the clock time accordingly.

20. Window Functions (Rolling, Expanding, EWM)

20.1 Rolling Windows

```
df['sales_7d_avg'] = df['sales'].rolling(window=7).mean()
df['sales_7d_sum'] = df['sales'].rolling(window=7).sum()
df['sales_7d_std'] = df['sales'].rolling(window=7, min_periods=1).std()
```

Code Line	Explanation
`.rolling(window=7)`	Defines a moving window of 7 consecutive rows; whatever aggregation is chained after it is computed fresh for each window position as it slides one row at a time down the Series.
`.rolling(window=7).mean()`	Calculates a 7-period moving average, a common technique for smoothing out day-to-day noise in a time series to reveal the underlying trend.
`min_periods=1`	Allows the calculation to produce a result even before a full window of 7 values is available (for example, on the very first few rows), using however many values are actually present instead of returning NaN until the window fully fills.

20.2 Expanding Windows

```
df['sales_cumulative_avg'] = df['sales'].expanding().mean()
df['sales_running_total'] = df['sales'].expanding().sum()
```

Code Line	Explanation
`.expanding()`	Unlike a rolling window, which has a fixed size, an expanding window grows to include every row from the very start of the Series up to the current row, making it well suited for running totals or cumulative averages.

20.3 Exponentially Weighted Windows

```
df['sales_ewm'] = df['sales'].ewm(span=7).mean()
```

An exponentially weighted moving average gives more weight to recent observations and progressively less weight to older ones, controlled by the span parameter, rather than treating every value inside a fixed window equally. This makes it more responsive to recent changes than a simple rolling average, while still smoothing out short-term noise.

20.4 Cumulative Functions

```
df['cumsum'] = df['sales'].cumsum()
df['cummax'] = df['sales'].cummax()
df['cummin'] = df['sales'].cummin()
```

Code Line	Explanation
`.cumsum()`	Returns a running total, where each row's value is the sum of every

Code Line	Explanation
	value up to and including that row.
`.cummax()`	Returns the running maximum value seen so far at each row.
`.cummin()`	Returns the running minimum value seen so far at each row.

21. Categorical Data and Memory Optimization

21.1 Why Use the Category Dtype

When a text column has a limited, repeating set of values, such as department names or product categories, storing it as the generic object dtype wastes memory, since pandas stores a full separate Python string object for every single row, even when the same string repeats thousands of times. The category dtype instead stores each unique value once, and each row holds only a small integer reference to that value, similar to how a database uses a lookup table.

21.2 Converting to Category

```
df['department'] = df['department'].astype('category')
df['department'].memory_usage(deep=True)
```

Code Line	Explanation
`astype('category')`	Converts the column from object (text) dtype to category dtype.
`.memory_usage(deep=True)`	Reports the actual memory footprint of the column in bytes; comparing this before and after the conversion demonstrates the savings directly.

21.3 Ordered Categories

```
sizes = pd.CategoricalDtype(categories=['Small', 'Medium', 'Large'], ordered=True)
df['t_shirt_size'] = df['t_shirt_size'].astype(sizes)
df[df['t_shirt_size'] > 'Small']
df.sort_values('t_shirt_size')
```

Code Line	Explanation
`pd.CategoricalDtype(categories=[...], ordered=True)`	Defines a custom categorical type with an explicit, meaningful ranking between the categories, rather than the arbitrary alphabetical order pandas would otherwise assume.
`df['t_shirt_size'] > 'Small'`	Once the ordering is defined, comparison operators work correctly according to that logical order (Small < Medium < Large), which would not be possible on a plain text column.
`df.sort_values('t_shirt_size')`	Sorts rows according to the defined category order (Small, then Medium, then Large) instead of alphabetical order (which would incorrectly put Large before Medium before Small).

21.4 Other Memory-Saving Techniques

- Downcast numeric columns with pd.to_numeric(..., downcast='integer'/'float') as shown in section 11.3.
- Drop columns that are not needed for the current analysis with df.drop(columns=[...]).
- Load only necessary columns using usecols= when reading a file, rather than loading everything and dropping columns afterward.
- Use df.info(memory_usage='deep') periodically to track memory usage as a pipeline develops.

22. Exporting Data

22.1 Writing to CSV

```
df.to_csv('cleaned_employees.csv', index=False)
df.to_csv('cleaned_employees.csv', index=False, columns=['name', 'department',
'salary'])
```

Code Line	Explanation
`df.to_csv('cleaned_employees.csv', index=False)`	Writes the DataFrame to a new CSV file; index=False prevents the row index from being written out as an unwanted extra column, which is almost always the desired behavior when the index is just the default row number.
`columns=['name', 'department', 'salary']`	Restricts the export to only the listed columns, in the given order, even if the DataFrame itself contains more columns.

22.2 Writing to Excel

```
df.to_excel('report.xlsx', sheet_name='Employees', index=False)

with pd.ExcelWriter('report.xlsx') as writer:
    df_sales.to_excel(writer, sheet_name='Sales', index=False)
    df_expenses.to_excel(writer, sheet_name='Expenses', index=False)
```

Code Line	Explanation
`df.to_excel('report.xlsx', sheet_name='Employees', index=False)`	Writes the DataFrame to a single worksheet named 'Employees' inside a new Excel workbook.
`pd.ExcelWriter('report.xlsx') as writer`	Opens a single Excel workbook that multiple DataFrames can be written into, each to its own named sheet, without overwriting the previous sheet; the with block ensures the file is properly saved and closed automatically.

22.3 Writing to JSON, Parquet and SQL

```
df.to_json('data.json', orient='records', indent=2)
df.to_parquet('data.parquet', engine='pyarrow', index=False)
df.to_sql('employees', con=engine, if_exists='replace', index=False)
```

Code Line	Explanation
`orient='records'`	Writes the JSON as a list of row-like objects, matching the most common shape expected by web APIs and JavaScript consumers.
`df.to_parquet(..., engine='pyarrow')`	Writes the DataFrame to the compressed, columnar Parquet format, which preserves exact data types and is far more space- and speed-efficient than CSV for large datasets.
`df.to_sql('employees', con=engine, if_exists='replace')`	Writes the DataFrame into a database table named 'employees' through an open SQLAlchemy connection; if_exists='replace' drops and recreates the table first, while 'append' would add rows to an existing table instead.

22.4 A Note on Round-Tripping Data Types

CSV is a plain text format and does not preserve data types; every value written out becomes text again, meaning a datetime or category column read back in from a saved CSV will need to be re-converted with pd.to_datetime() or .astype('category') just as if it were a brand-new file. Parquet, by contrast, stores the exact data type of every column, so re-reading a Parquet file restores datetimes, categories and integer types exactly as they were, which makes it the more reliable choice for intermediate files in a multi-step pipeline.

23. Performance Tips and Best Practices

23.1 The Golden Rule: Vectorize

Every performance recommendation in pandas ultimately traces back to one idea: let pandas and numpy operate on whole arrays at once in compiled code, rather than writing an explicit Python for loop or .apply() that processes one row at a time. The difference in speed, especially on large datasets, is not marginal; it is very commonly a 10 to 100 times speedup.

```
# Slow: explicit Python loop
results = []
for i in range(len(df)):
    results.append(df['price'].iloc[i] * df['quantity'].iloc[i])
df['revenue'] = results

# Fast: vectorized
df['revenue'] = df['price'] * df['quantity']
```

23.2 Avoid Growing a DataFrame Row by Row

```
# Slow: repeated concat/append inside a loop
df = pd.DataFrame()
for record in records:
    df = pd.concat([df, pd.DataFrame([record])], ignore_index=True)

# Fast: build a list of records, then create the DataFrame once
rows = [record for record in records]
df = pd.DataFrame(rows)
```

Each call to pd.concat() inside a loop copies the entire growing DataFrame in memory, so repeating it n times costs roughly O(n squared) time overall. Collecting plain Python dictionaries or lists first, and constructing the DataFrame only once at the end, avoids this repeated copying entirely.

23.3 Use Efficient Data Types

- Convert repeating text columns to category dtype (section 21).
- Downcast numeric columns to the smallest safe integer or float type (section 11.3).
- Load only the columns actually needed with usecols= when reading a file (section 5.1).
- Consider the newer nullable dtypes (Int64, Float64, boolean, string) when a column needs to support pandas' improved missing-value handling for integers, which the traditional numpy-backed int64 cannot represent NaN in.

23.4 Filter Early, Filter Often

```
df_filtered = df[df['status'] == 'active']
result = df_filtered.groupby('department')['salary'].mean()
```

Filtering the DataFrame down to only the rows actually needed before running an expensive operation like groupby(), merge() or apply() reduces the amount of data that operation has to process, often producing a significant speedup with no change in the final result.

23.5 Profiling and Timing

```
%timeit df['salary'] * 1.1

import time
start = time.time()
df.groupby('department')['salary'].mean()
print(f'Elapsed: {time.time() - start:.4f} seconds')
```

Code Line	Explanation
`%timeit`	A Jupyter magic command that runs the given line multiple times automatically and reports the average execution time, useful for quickly comparing two alternative approaches to the same task.
`time.time()`	A plain Python approach that works outside notebooks too: recording a timestamp before and after an operation and subtracting to measure elapsed wall-clock time.

24. Full Worked Example (Line-by-Line, End-to-End)

This section applies the techniques from every previous section to one continuous, realistic scenario: cleaning, transforming and summarizing a retail orders dataset.

24.1 Step 1: Imports and Load

```
import pandas as pd
import numpy as np

pd.set_option('display.max_columns', None)

df = pd.read_csv('orders.csv', parse_dates=['order_date'])
print(df.shape)
df.head()
```

Loads the toolkit, sets a display option so wide previews are not truncated, reads the raw CSV while immediately parsing order_date as a real datetime, then confirms the size and previews the first rows.

24.2 Step 2: Structural Check

```
df.info()
df.isnull().sum().sort_values(ascending=False)
df.duplicated().sum()
```

Checks data types and non-null counts, ranks columns by missing data, and counts fully duplicated rows, forming a quick data-quality snapshot before any cleaning decisions are made.

24.3 Step 3: Cleaning

```
df = df.drop_duplicates()
df['unit_price'] = pd.to_numeric(df['unit_price'], errors='coerce')
df['unit_price'] = df['unit_price'].fillna(df.groupby('product_category')
['unit_price'].transform('median'))
df = df.dropna(subset=['order_date', 'customer_id'])
df['region'] = df['region'].str.strip().str.title()
df['product_category'] = df['product_category'].astype('category')
```

Duplicates are removed first so they cannot inflate later sums. The unit_price column is coerced to numeric, then any resulting gaps are filled using each product category's own median price, a more precise imputation than a single global median. Rows missing a critical identifier are dropped, region text is standardized, and the category column is converted to the memory-efficient category dtype.

24.4 Step 4: Feature Engineering

```
df['revenue'] = df['unit_price'] * df['quantity']
df['order_month'] = df['order_date'].dt.to_period('M')
df['is_weekend'] = df['order_date'].dt.dayofweek >= 5
```

Computes total revenue per order line with a fully vectorized multiplication, extracts a month period for later monthly aggregation, and flags weekend orders using the day-of-week number.

24.5 Step 5: GroupBy Summary

```
monthly_summary = df.groupby('order_month').agg(
    total_revenue=('revenue', 'sum'),
    orders=('order_id', 'nunique'),
    avg_order_value=('revenue', 'mean')
)
monthly_summary
```

Uses named aggregation to build a clean monthly summary table in one call: total revenue, distinct order count, and average order value, each computed from a different source column.

24.6 Step 6: Merge in Customer Data

```
customers = pd.read_csv('customers.csv')
df = pd.merge(df, customers[['customer_id', 'signup_date', 'segment']],
on='customer_id', how='left')
```

Reads a second reference file and left-joins two extra customer attributes onto the orders table, keeping every order row even if a matching customer record is somehow missing.

24.7 Step 7: Reshape for Reporting

```
pivot = pd.pivot_table(
    df,
    values='revenue',
    index='region',
    columns='product_category',
    aggfunc='sum',
    fill_value=0
)
pivot
```

Builds a region-by-category revenue pivot table, spreading the product_category values out into columns for an easy-to-scan spreadsheet-style summary.

24.8 Step 8: Save the Results

```
df.to_parquet('orders_cleaned.parquet', index=False)
monthly_summary.to_csv('monthly_summary.csv')
pivot.to_excel('region_category_pivot.xlsx')
```

Saves the full cleaned transaction-level data in Parquet (preserving exact data types for later reuse), and exports the two summary tables to CSV and Excel respectively, ready to share with non-technical stakeholders.

25. Common Pitfalls and How to Avoid Them

25.1 Frequent Mistakes

Code Line	Explanation
`Chained indexing when assigning values`	Writing df[df['x'] > 0]['y'] = 1 can silently fail to update the original DataFrame and triggers a SettingWithCopyWarning; use df.loc[df['x'] > 0, 'y'] = 1 instead.
`Forgetting errors='coerce' on type conversions`	pd.to_numeric() or pd.to_datetime() raise an error and halt execution on the first unparseable value unless errors='coerce' is set.
`Using .loc and .iloc slicing interchangeably`	.loc includes the endpoint of a slice; .iloc excludes it, matching standard Python slicing. Mixing up the two silently changes which rows are included.
`Comparing a Series to another Series without matching indexes`	Two Series with different or misaligned indexes will produce NaN in the mismatched positions during comparison or arithmetic, rather than an error, which can silently corrupt a calculation.
`Assuming groupby preserves the original row order`	groupby().apply() and similar operations can reorder rows by group key; call .sort_index() or explicitly sort afterward if original order matters.
`Growing a DataFrame with repeated concat() or loc-based appends inside a loop`	This is quadratically slow; collect records in a plain Python list and build the DataFrame once, as shown in section 23.2.
`Not checking for duplicate rows before merging`	A many-to-many merge on a key with unexpected duplicates on either side silently multiplies the row count (a 'merge explosion'), inflating any subsequent sums or counts.
`Forgetting that inplace=True still requires reassignment awareness`	Many pandas methods return a new object by default rather than modifying in place; inplace=True modifies the object directly but returns None, so writing df = df.dropna(inplace=True) silently sets df to None.

25.2 Verifying a Merge Did Not Explode Row Counts

```
print(len(employees), len(salaries))
merged = pd.merge(employees, salaries, on='emp_id', how='left')
print(len(merged))
assert len(merged) == len(employees), 'Merge changed row count unexpectedly'
```

Printing and comparing row counts before and after a merge is a fast sanity check; an unexpected increase in row count after a left join usually means the right-hand table has duplicate keys, which is worth investigating before trusting any downstream aggregation.

26. Quick-Reference Cheat Sheet

26.1 Inspection

Code Line	Explanation
`df.shape / df.info() / df.describe()`	Size, structure and summary statistics.
`df.head() / df.tail() / df.sample()`	Preview rows.
`df.dtypes / df.columns / df.index`	Column types, column names, row labels.

26.2 Selecting and Filtering

Code Line	Explanation
`df['col'] / df[['a', 'b']]`	Select one or more columns.
`df.loc[rows, cols] / df.iloc[rows, cols]`	Label-based / position-based selection.
`df[df['col'] > x]`	Boolean filtering.
`df.query('col > 5')`	String-based filtering.
`df['col'].isin([...])`	Membership filtering.

26.3 Cleaning

Code Line	Explanation
`df.isnull().sum() / df.dropna() / df.fillna(x)`	Detect, drop, or fill missing values.
`df.duplicated() / df.drop_duplicates()`	Detect and remove duplicate rows.
`df['col'].astype(type)`	Convert a column's data type.
`pd.to_datetime() / pd.to_numeric()`	Parse text into dates or numbers.

26.4 Transforming

Code Line	Explanation
`df['col'].apply(func)`	Apply a function element-wise.
`np.where(cond, a, b) / np.select(conds, choices)`	Fast vectorized conditional logic.
`df['col'].map(dict_or_func)`	Value substitution or lookup.
`df['col'].str.method()`	Vectorized string operations.

26.5 Grouping, Merging, Reshaping

Code Line	Explanation
`df.groupby('col').agg(...)`	Split-apply-combine aggregation.
`df.groupby('col')['x'].transform(func)`	Group result broadcast back to every row.
`pd.merge(a, b, on='key', how='...')`	SQL-style join between two tables.
`pd.concat([a, b])`	Stack DataFrames by rows or columns.
`pd.pivot_table(df, values, index, columns, aggfunc)`	Long to wide reshape.
`pd.melt(df, id_vars,`	Wide to long reshape.

Code Line	Explanation
`value_vars)`	

26.6 Time Series

Code Line	Explanation
`df.set_index('date_col')`	Enable time-series-specific indexing.
`df.resample('M').sum()`	Aggregate into fixed time buckets.
`df['col'].rolling(window=n).mean()`	Moving average over a fixed window.
`df['col'].shift(1) / .diff() / .pct_change()`	Lag, difference, percent change.

26.7 Exporting

Code Line	Explanation
`df.to_csv(path, index=False)`	Write to CSV.
`df.to_excel(path, index=False)`	Write to Excel.
`df.to_parquet(path, index=False)`	Write to Parquet (preserves dtypes).

27. Conclusion

Pandas turns the mechanics of working with tabular data, loading, cleaning, filtering, transforming, aggregating and reshaping, into a small set of composable, well-tested building blocks. Nearly every real-world data task in Python, from a five-minute CSV cleanup to a production data pipeline, is built from combinations of the techniques covered in this guide: selecting with .loc and .iloc, cleaning with dropna and fillna, transforming with vectorized arithmetic instead of loops, summarizing with groupby, combining tables with merge and concat, and reshaping with pivot_table and melt.

The line-by-line breakdowns throughout are meant to remove any ambiguity about what each piece of code actually does, so the same patterns can be confidently adapted to a new dataset with different column names and a different shape. The single habit most worth carrying forward is to reach for a vectorized, built-in pandas operation before writing a manual loop; pandas was built from the ground up to make that the fast path, not the exception.

Used together with the Exploratory Data Analysis techniques from the companion guide, pandas provides everything needed to take a raw, messy file and turn it into a clean, well-understood dataset ready for analysis, visualization or modeling.

End of Document

Kajola Gbenga | Data Scientist & AI Engineer